

TECHNICAL LEARNING FILE



DEEP-10

Exploratory Analysis and Visualization

Evidence-first data communication

FORMAT	LENGTH	ACCESS
INTENSIVE FILE	50 PAGES	OFFLINE

Edition 2 - original curriculum - editorial review recommended



FILE / ORIENTATION

Purpose

01 FILE GUIDANCE

This optional advanced guide does not gate the core learning path. Apply exploratory analysis and visualization with explicit assumptions, tests, tradeoffs, and limitations.

02 LOCAL USE

Index this page in the offline database and preserve its resource and page identifiers for bookmarks, notes, highlights, and progress.



FILE / ORIENTATION

Prerequisites

01 FILE GUIDANCE

Complete the related core track or pass its diagnostic.

NOUS AI ACADEMY

02 LOCAL USE

Index this page in the offline database and preserve its resource and page identifiers for bookmarks, notes, highlights, and progress.



FILE / ORIENTATION

Study Method

01 FILE GUIDANCE

For each unit, explain the concept, follow the workflow, reproduce the example, analyze a failure, and complete the applied lab. Record evidence locally.

02 LOCAL USE

Index this page in the offline database and preserve its resource and page identifiers for bookmarks, notes, highlights, and progress.



UNIT 01 / CONCEPT

Question Design: Concept

01 OBJECTIVE

Explain and apply question design using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Question Design is a central part of exploratory analysis and visualization because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Exploratory Analysis and Visualization, the terms Question, Design are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should question design be used, and what observable evidence would make that choice defensible? Build a small local example that applies question design, records the result, and tests one failure condition.

04 IMPLEMENTATION

define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations

05 PRACTICE

1. Define Question Design in one precise sentence and name one assumption.
2. Create a two-step example of question design and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 01 / WORKFLOW

Question Design: Workflow

01 OBJECTIVE

Explain and apply question design using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Question Design is a central part of exploratory analysis and visualization because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to question design.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies question design, records the result, and tests one failure condition.

04 IMPLEMENTATION

define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations

05 PRACTICE

1. Define Question Design in one precise sentence and name one assumption.
2. Create a two-step example of question design and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 01 / WORKED EXAMPLE

Question Design: Worked Example

01 OBJECTIVE

Explain and apply question design using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies question design, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns question design into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations

05 PRACTICE

1. Define Question Design in one precise sentence and name one assumption.
2. Create a two-step example of question design and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 01 / FAILURE ANALYSIS

Question Design: Failure Analysis

01 OBJECTIVE

Explain and apply question design using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how question design can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to question design. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations

05 PRACTICE

1. Define Question Design in one precise sentence and name one assumption.
2. Create a two-step example of question design and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 01 / APPLIED LAB

Question Design: Applied Lab

01 OBJECTIVE

Explain and apply question design using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of question design into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies question design, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations

05 PRACTICE

1. Define Question Design in one precise sentence and name one assumption.
2. Create a two-step example of question design and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 02 / CONCEPT

Distribution Views: Concept

01 OBJECTIVE

Explain and apply distribution views using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Distribution Views is a central part of exploratory analysis and visualization because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Exploratory Analysis and Visualization, the terms Distribution, Views are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should distribution views be used, and what observable evidence would make that choice defensible? Build a small local example that applies distribution views, records the result, and tests one failure condition.

04 IMPLEMENTATION

define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations

05 PRACTICE

1. Define Distribution Views in one precise sentence and name one assumption.
2. Create a two-step example of distribution views and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 02 / WORKFLOW

Distribution Views: Workflow

01 OBJECTIVE

Explain and apply distribution views using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Distribution Views is a central part of exploratory analysis and visualization because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to distribution views.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies distribution views, records the result, and tests one failure condition.

04 IMPLEMENTATION

define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations

05 PRACTICE

1. Define Distribution Views in one precise sentence and name one assumption.
2. Create a two-step example of distribution views and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 02 / WORKED EXAMPLE

Distribution Views: Worked Example

01 OBJECTIVE

Explain and apply distribution views using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies distribution views, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns distribution views into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations

05 PRACTICE

1. Define Distribution Views in one precise sentence and name one assumption.
2. Create a two-step example of distribution views and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 02 / FAILURE ANALYSIS

Distribution Views: Failure Analysis

01 OBJECTIVE

Explain and apply distribution views using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how distribution views can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to distribution views. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations

05 PRACTICE

1. Define Distribution Views in one precise sentence and name one assumption.
2. Create a two-step example of distribution views and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 02 / APPLIED LAB

Distribution Views: Applied Lab

01 OBJECTIVE

Explain and apply distribution views using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of distribution views into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies distribution views, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations

05 PRACTICE

1. Define Distribution Views in one precise sentence and name one assumption.
2. Create a two-step example of distribution views and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 03 / CONCEPT

Relationship Views: Concept

01 OBJECTIVE

Explain and apply relationship views using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Relationship Views is a central part of exploratory analysis and visualization because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Exploratory Analysis and Visualization, the terms Relationship Views are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should relationship views be used, and what observable evidence would make that choice defensible? Build a small local example that applies relationship views, records the result, and tests one failure condition.

04 IMPLEMENTATION

define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations

05 PRACTICE

1. Define Relationship Views in one precise sentence and name one assumption.
2. Create a two-step example of relationship views and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 03 / WORKFLOW

Relationship Views: Workflow

01 OBJECTIVE

Explain and apply relationship views using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Relationship Views is a central part of exploratory analysis and visualization because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to relationship views.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies relationship views, records the result, and tests one failure condition.

04 IMPLEMENTATION

define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations

05 PRACTICE

1. Define Relationship Views in one precise sentence and name one assumption.
2. Create a two-step example of relationship views and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 03 / WORKED EXAMPLE

Relationship Views: Worked Example

01 OBJECTIVE

Explain and apply relationship views using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies relationship views, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns relationship views into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations

05 PRACTICE

1. Define Relationship Views in one precise sentence and name one assumption.
2. Create a two-step example of relationship views and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 03 / FAILURE ANALYSIS

Relationship Views: Failure Analysis

01 OBJECTIVE

Explain and apply relationship views using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how relationship views can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to relationship views. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations

05 PRACTICE

1. Define Relationship Views in one precise sentence and name one assumption.
2. Create a two-step example of relationship views and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 03 / APPLIED LAB

Relationship Views: Applied Lab

01 OBJECTIVE

Explain and apply relationship views using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of relationship views into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies relationship views, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations

05 PRACTICE

1. Define Relationship Views in one precise sentence and name one assumption.
2. Create a two-step example of relationship views and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 04 / CONCEPT

Uncertainty: Concept

01 OBJECTIVE

Explain and apply uncertainty using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Uncertainty is a central part of exploratory analysis and visualization because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Exploratory Analysis and Visualization, the terms Uncertainty are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should uncertainty be used, and what observable evidence would make that choice defensible? Build a small local example that applies uncertainty, records the result, and tests one failure condition.

04 IMPLEMENTATION

define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations

05 PRACTICE

1. Define Uncertainty in one precise sentence and name one assumption.
2. Create a two-step example of uncertainty and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 04 / WORKFLOW

Uncertainty: Workflow

01 OBJECTIVE

Explain and apply uncertainty using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Uncertainty is a central part of exploratory analysis and visualization because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to uncertainty.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies uncertainty, records the result, and tests one failure condition.

04 IMPLEMENTATION

define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations

05 PRACTICE

1. Define Uncertainty in one precise sentence and name one assumption.
2. Create a two-step example of uncertainty and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 04 / WORKED EXAMPLE

Uncertainty: Worked Example

01 OBJECTIVE

Explain and apply uncertainty using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies uncertainty, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns uncertainty into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations

05 PRACTICE

1. Define Uncertainty in one precise sentence and name one assumption.
2. Create a two-step example of uncertainty and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 04 / FAILURE ANALYSIS

Uncertainty: Failure Analysis

01 OBJECTIVE

Explain and apply uncertainty using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how uncertainty can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to uncertainty. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations

05 PRACTICE

1. Define Uncertainty in one precise sentence and name one assumption.
2. Create a two-step example of uncertainty and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 04 / APPLIED LAB

Uncertainty: Applied Lab

01 OBJECTIVE

Explain and apply uncertainty using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of uncertainty into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies uncertainty, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations

05 PRACTICE

1. Define Uncertainty in one precise sentence and name one assumption.
2. Create a two-step example of uncertainty and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 05 / CONCEPT

Color and Accessibility: Concept

01 OBJECTIVE

Explain and apply color and accessibility using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Color and Accessibility is a central part of exploratory analysis and visualization because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Exploratory Analysis and Visualization, the terms Color, Accessibility are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should color and accessibility be used, and what observable evidence would make that choice defensible? Build a small local example that applies color and accessibility, records the result, and tests one failure condition.

04 IMPLEMENTATION

define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations

05 PRACTICE

1. Define Color and Accessibility in one precise sentence and name one assumption.
2. Create a two-step example of color and accessibility and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 05 / WORKFLOW

Color and Accessibility: Workflow

01 OBJECTIVE

Explain and apply color and accessibility using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Color and Accessibility is a central part of exploratory analysis and visualization because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to color and accessibility.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies color and accessibility, records the result, and tests one failure condition.

04 IMPLEMENTATION

define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations

05 PRACTICE

1. Define Color and Accessibility in one precise sentence and name one assumption.
2. Create a two-step example of color and accessibility and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 05 / WORKED EXAMPLE

Color and Accessibility: Worked Example

01 OBJECTIVE

Explain and apply color and accessibility using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies color and accessibility, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns color and accessibility into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations

05 PRACTICE

1. Define Color and Accessibility in one precise sentence and name one assumption.
2. Create a two-step example of color and accessibility and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 05 / FAILURE ANALYSIS

Color and Accessibility: Failure Analysis

01 OBJECTIVE

Explain and apply color and accessibility using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how color and accessibility can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to color and accessibility. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations

05 PRACTICE

1. Define Color and Accessibility in one precise sentence and name one assumption.
2. Create a two-step example of color and accessibility and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 05 / APPLIED LAB

Color and Accessibility: Applied Lab

01 OBJECTIVE

Explain and apply color and accessibility using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of color and accessibility into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies color and accessibility, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations

05 PRACTICE

1. Define Color and Accessibility in one precise sentence and name one assumption.
2. Create a two-step example of color and accessibility and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 06 / CONCEPT

Small Multiples: Concept

01 OBJECTIVE

Explain and apply small multiples using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Small Multiples is a central part of exploratory analysis and visualization because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Exploratory Analysis and Visualization, the terms Small, Multiples are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should small multiples be used, and what observable evidence would make that choice defensible? Build a small local example that applies small multiples, records the result, and tests one failure condition.

04 IMPLEMENTATION

define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations

05 PRACTICE

1. Define Small Multiples in one precise sentence and name one assumption.
2. Create a two-step example of small multiples and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 06 / WORKFLOW

Small Multiples: Workflow

01 OBJECTIVE

Explain and apply small multiples using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Small Multiples is a central part of exploratory analysis and visualization because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to small multiples.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies small multiples, records the result, and tests one failure condition.

04 IMPLEMENTATION

define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations

05 PRACTICE

1. Define Small Multiples in one precise sentence and name one assumption.
2. Create a two-step example of small multiples and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 06 / WORKED EXAMPLE

Small Multiples: Worked Example

01 OBJECTIVE

Explain and apply small multiples using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies small multiples, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns small multiples into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations

05 PRACTICE

1. Define Small Multiples in one precise sentence and name one assumption.
2. Create a two-step example of small multiples and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 06 / FAILURE ANALYSIS

Small Multiples: Failure Analysis

01 OBJECTIVE

Explain and apply small multiples using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how small multiples can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to small multiples. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations

05 PRACTICE

1. Define Small Multiples in one precise sentence and name one assumption.
2. Create a two-step example of small multiples and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 06 / APPLIED LAB

Small Multiples: Applied Lab

01 OBJECTIVE

Explain and apply small multiples using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of small multiples into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies small multiples, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations

05 PRACTICE

1. Define Small Multiples in one precise sentence and name one assumption.
2. Create a two-step example of small multiples and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 07 / CONCEPT

Interactive Views: Concept

01 OBJECTIVE

Explain and apply interactive views using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Interactive Views is a central part of exploratory analysis and visualization because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Exploratory Analysis and Visualization, the terms Interactive Views are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should interactive views be used, and what observable evidence would make that choice defensible? Build a small local example that applies interactive views, records the result, and tests one failure condition.

04 IMPLEMENTATION

define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations

05 PRACTICE

1. Define Interactive Views in one precise sentence and name one assumption.
2. Create a two-step example of interactive views and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 07 / WORKFLOW

Interactive Views: Workflow

01 OBJECTIVE

Explain and apply interactive views using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Interactive Views is a central part of exploratory analysis and visualization because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to interactive views.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies interactive views, records the result, and tests one failure condition.

04 IMPLEMENTATION

define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations

05 PRACTICE

1. Define Interactive Views in one precise sentence and name one assumption.
2. Create a two-step example of interactive views and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 07 / WORKED EXAMPLE

Interactive Views: Worked Example

01 OBJECTIVE

Explain and apply interactive views using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies interactive views, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns interactive views into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations

05 PRACTICE

1. Define Interactive Views in one precise sentence and name one assumption.
2. Create a two-step example of interactive views and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 07 / FAILURE ANALYSIS

Interactive Views: Failure Analysis

01 OBJECTIVE

Explain and apply interactive views using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how interactive views can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to interactive views. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations

05 PRACTICE

1. Define Interactive Views in one precise sentence and name one assumption.
2. Create a two-step example of interactive views and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 07 / APPLIED LAB

Interactive Views: Applied Lab

01 OBJECTIVE

Explain and apply interactive views using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of interactive views into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies interactive views, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations

05 PRACTICE

1. Define Interactive Views in one precise sentence and name one assumption.
2. Create a two-step example of interactive views and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 08 / CONCEPT

Misleading Charts: Concept

01 OBJECTIVE

Explain and apply misleading charts using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Misleading Charts is a central part of exploratory analysis and visualization because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Exploratory Analysis and Visualization, the terms Misleading, Charts are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should misleading charts be used, and what observable evidence would make that choice defensible? Build a small local example that applies misleading charts, records the result, and tests one failure condition.

04 IMPLEMENTATION

define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations

05 PRACTICE

1. Define Misleading Charts in one precise sentence and name one assumption.
2. Create a two-step example of misleading charts and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 08 / WORKFLOW

Misleading Charts: Workflow

01 OBJECTIVE

Explain and apply misleading charts using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Misleading Charts is a central part of exploratory analysis and visualization because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to misleading charts.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies misleading charts, records the result, and tests one failure condition.

04 IMPLEMENTATION

define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations

05 PRACTICE

1. Define Misleading Charts in one precise sentence and name one assumption.
2. Create a two-step example of misleading charts and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 08 / WORKED EXAMPLE

Misleading Charts: Worked Example

01 OBJECTIVE

Explain and apply misleading charts using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies misleading charts, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns misleading charts into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations

05 PRACTICE

1. Define Misleading Charts in one precise sentence and name one assumption.
2. Create a two-step example of misleading charts and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 08 / FAILURE ANALYSIS

Misleading Charts: Failure Analysis

01 OBJECTIVE

Explain and apply misleading charts using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how misleading charts can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to misleading charts. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations

05 PRACTICE

1. Define Misleading Charts in one precise sentence and name one assumption.
2. Create a two-step example of misleading charts and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 08 / APPLIED LAB

Misleading Charts: Applied Lab

01 OBJECTIVE

Explain and apply misleading charts using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of misleading charts into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies misleading charts, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations

05 PRACTICE

1. Define Misleading Charts in one precise sentence and name one assumption.
2. Create a two-step example of misleading charts and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 09 / CONCEPT

Narrative Reports: Concept

01 OBJECTIVE

Explain and apply narrative reports using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Narrative Reports is a central part of exploratory analysis and visualization because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Exploratory Analysis and Visualization, the terms Narrative, Reports are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should narrative reports be used, and what observable evidence would make that choice defensible? Build a small local example that applies narrative reports, records the result, and tests one failure condition.

04 IMPLEMENTATION

define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations

05 PRACTICE

1. Define Narrative Reports in one precise sentence and name one assumption.
2. Create a two-step example of narrative reports and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 09 / WORKFLOW

Narrative Reports: Workflow

01 OBJECTIVE

Explain and apply narrative reports using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Narrative Reports is a central part of exploratory analysis and visualization because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to narrative reports.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies narrative reports, records the result, and tests one failure condition.

04 IMPLEMENTATION

define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations

05 PRACTICE

1. Define Narrative Reports in one precise sentence and name one assumption.
2. Create a two-step example of narrative reports and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 09 / WORKED EXAMPLE

Narrative Reports: Worked Example

01 OBJECTIVE

Explain and apply narrative reports using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies narrative reports, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns narrative reports into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations

05 PRACTICE

1. Define Narrative Reports in one precise sentence and name one assumption.
2. Create a two-step example of narrative reports and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 09 / FAILURE ANALYSIS

Narrative Reports: Failure Analysis

01 OBJECTIVE

Explain and apply narrative reports using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how narrative reports can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to narrative reports. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations

05 PRACTICE

1. Define Narrative Reports in one precise sentence and name one assumption.
2. Create a two-step example of narrative reports and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 09 / APPLIED LAB

Narrative Reports: Applied Lab

01 OBJECTIVE

Explain and apply narrative reports using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of narrative reports into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies narrative reports, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

define inputs and expected outputs
state assumptions
implement the smallest baseline
test normal and edge cases
record limitations

05 PRACTICE

1. Define Narrative Reports in one precise sentence and name one assumption.
2. Create a two-step example of narrative reports and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



FILE / ORIENTATION

Deep-Dive Capstone

01 FILE GUIDANCE

Build a compact, reproducible artifact demonstrating exploratory analysis and visualization. Include assumptions, a baseline, tests, failure analysis, results, limitations, and instructions for repeating the work offline.

02 LOCAL USE

Index this page in the offline database and preserve its resource and page identifiers for bookmarks, notes, highlights, and progress.